

Polymorphism

Polymorphism is the ability of an object to take on different forms depending on the context. In programming, it usually means having a single method name that does fundamentally different things depending on which object is using it.

- **The Concept:** At a high level of abstraction, calculating an **"Area"** is the same idea. But at the low level of implementation, the **Area()** of a **Rectangle (length * width)** requires completely different math than the **Area()** of a **Triangle (0.5 * base * height)**.
- Polymorphism allows us to just call **.Area()** on any shape, and trust that the shape knows exactly which math to run.

1. How Traditional OOP Does It: Inheritance

In languages like Python or C++, polymorphism is achieved through **Inheritance**.

- **The Traditional Setup:** You create a **"Parent"** class (Superclass) and **"Child"** classes (Subclasses). The children automatically inherit all the data and methods of the parent.
- **Example:** You create a **Speaker** superclass with a method called **Speak()** that just prints **"noise"**. You then create a **Cat** subclass and a **Dog** subclass. Both animals automatically inherit the **Speak()** method.

Note: Go DOES NOT have inheritance.

2. The Power of Overriding

If a Dog inherits the **Speak()** method from **Speaker**, it will print **"noise"**. To make it polymorphic, traditional OOP uses **Overriding**.

- **What is it?** Overriding is when a subclass intentionally redefines a method it inherited from its parent.
- **The Rules:** The method **Signature** (the **name**, the **inputs**, and the **return types**) must remain exactly the same.
- **In Action:** The **Cat** class overrides **Speak()** to print **"meow"**. The **Dog** class overrides **Speak()** to print **"woof"**.

Now, **Speak()** is truly polymorphic. You can tell any **Speaker** to **Speak()**, and it will execute entirely different code depending on whether it is currently taking the form of a **Cat** or a **Dog**.

Expert Takeaway:

Why doesn't Go use Inheritance? Inheritance creates rigid, deeply nested **"family trees"** of code that become incredibly fragile in massive enterprise architectures. If you change a method in the Parent class, it ripples down and can accidentally break **50** Child classes. Go takes a much more modern, decoupled approach using **Composition** and **Interfaces**.